

Dynamic Memory Allocation in C (malloc, calloc, realloc, free)

> **Definition to Remember**

1. Concept Overview & Fundamentals

Dynamic Memory Allocation in C (malloc, calloc, realloc, free) forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization

```text

### Core Architecture Breakdown

| Component / Concept | Technical Specification | Operational Characteristics | Performance / Complexity |

| :--- | :

---

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
```c
// Core standard implementation pattern
// Core logic execution
return status;
```
#include
int mai
printf("
}
```

---

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

- **E**

---

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: calloc sentence

**Problem Description & Objective:**

**\*\*C Code Implementation:\*\***

```
int main() {
```

```
int size;
```

```
char *str = (char *)calloc(size, sizeof(char));
```

```
if (str == NULL) {
```

```
getchar(); // clear newline
```

```
printf("Enter sentence: ");
```

```
printf("You entered: %s", str);
```

```
free(str);
```

```

```

## **Problem 2: car malloc**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
struct Car {
```

```
int main() {
```

$$J^*$$

```
struct Car *c;
```

```
// allocate memory
```

$c = (\text{str}$

```
if (c == NULL) {
```

```
printf("
```

```
// assign values
```

```
strcpy(
```

```
// print
```

```
printf("
```

```
// free memory
```

```
free(c)
```

```
return 0;
```

}

...

### Problem 3: float array malloc

### **\*\*Problem Description & Objective:\*\***

## Implement

### **\*\*C Code Implementation:\*\***

 $\text{C}$ 

```
int main() {
```

 $\beta^*$

```
int n;
```

```
printf(''
```

```
float *arr = (float *)malloc(n * sizeof(float));
```

```
if (arr == NULL) {
```

```
printf("
```

```
// input
```

```
for (int
```

```
// output
```

```
printf("
```

```
free(arr);
```

return

— — —

## Problem 4: point struct malloc

### **\*\*Problem Description & Objective:\*\***

## Implement

### **\*\*C Code Implementation:\*\***

 ${}'''C$ 

```
struct Point {
```

```
int x;
```

```
int main() {
```

 $\rho^*$ 

```
struct Point *p;
```

```
p = (struct Point *)malloc(sizeof(struct Point));
```

```
if (p == NULL) {
```

```
printf("p == NULL\n");
```

```
// assign values
```

```
p->x = 10;
```

```
printf("Point: (%d, %d)\n", p->x, p->y);
```

```
free(p);
```

```
return 0;
```

```
}
```

```

```

### Problem 5: realloc shrink

**\*\*Problem Description & Objective:\*\***

Implement the following program to demonstrate the use of realloc function to shrink the memory.

**\*\*C Code Implementation:\*\***

```
```c
```

```
int main() {
```

```
/*
```

```
int n;
```

```
printf("Enter number of elements: ");
```

```
scanf("%d", &n);
```

```
int *arr = (int *)calloc(n, sizeof(int));
```

```
if (arr == NULL) {
```

```
printf("Memory allocation failed\n");
```

```
// random numbers
```

```
srand((int)time(NULL));
```

```
printf("Original array:\n");
```

```
for (int
```

```
// shrink size
```

```
int new
```

```
printf("\n\nAfter shrinking:\n");
```

```
for (int
```

```
free(arr);
```

```
return
```

```
---
```

6. Exam Scoring Points & Memory Revision

> **Must-Write Points**

> 1. m

> **Quick Recall**

> `mal